Take-Home Exercise — *"Docufi Mini-Chat"*

Goal show that you can ① accept a finance document, ② extract a few hard facts, and ③ expose a single chat endpoint that can reference those facts when the user asks questions. Total effort ~4h for a junior/mid engineer.

1. **Upload** – upload documents and store them in an embedding format.
2. **Extract key facts** – pull out key data if needed, .ie.: *Revenue*, *EBITDA*, and *YoY growth* **only if they appear verbatim** (no calculations).
3. **Use an LLM** for (1) and (3); for (2) you may use an LLM to produce results feel free to add agents or processes you consider to improve results. No arithmetic required.

Material

💭Use /documents EA's material to work and test.

Minimal Functional Requirements

#	Endpoint	Method & Path	Purpose	Payload ↔ Response
1	**Upload**	`POST /upload`	Store one PDF / DOCX / plain-text file. Extract *verbatim* facts if present: **Revenue**, **EBITDA**, **YoY growth**.	**Request** `multipart/form-data` (`file`) **Response** `{ "docId": "<uuid>" }`
2	**Conversation**	`POST /conversation`	Simple chat interface tied to a `docId`. Messages may ask free-form questions; the assistant should answer and *inject* any extracted facts when relevant ("Revenue was $123 M in FY-24").	**Request** `json { "docId": "<uuid>", "message": "string" }` **Response** `json { "reply": "string" }`

Return JSON, proper HTTP status codes (400 bad input, 404 unknown `docId`, 5xx errors).

If you have time.

!!Extra points if you implement a way to outline content for a given slide topic, ie.: “Competitive Advantages” > Returns > “Company’s competitive advantages are…”.

!!No need for a UI… but feel free to flex.

!!Simple is always better, feel free to solve the exercise in your favourite language or framework.

Beware

⚠️**We allow code copilot usage to complete the exercise**, actually we encourage it if done smartly (ie.: edge cases, tests, Docker, Makefile… just a few ideas). We will talk about the code so don't let the LLM do the heavy thinking.